Adjoints revisited

Soham Saha

https://soham-saha.github.io

https://soham-saha.codeberg.page

In the previous note on adjoints we saw how an adjoint can be thought of as particular hom-set bijection. Here, we show how this definition relates to the universal morphism based definition and the unit-counit definition.

We shall also demonstrate a startling lack of proofs of any sort, relying on the unfortunate reader to work them out themselves.

Throughout this note, assume that we have categories $\boldsymbol{C}, \boldsymbol{D}$, and a functor $R : \boldsymbol{C} \to \boldsymbol{D}$.

First, observe that if we have $L \dashv R$ for some functor $L : \boldsymbol{D} \to \boldsymbol{C}$, then we have a universal morphism from every $X \in \mathrm{obj}(\boldsymbol{D})$ to the functor R; namely $(L(X), i_X : X \to RL(X))$, where i_X is $\mathrm{id}_{L(X)}$ carried through the hom-set bijection given by the adjoint property.

Now, we ask two questions:

1. Is the presence of universal morphisms from every $X \in \mathrm{obj}(\boldsymbol{D})$ to R enough to guarantee a left adjoint functor for R?
2. What universal property does a left adjoint of R satisfy?

Let us resolve these questions one by one.

For the first question, we assume that we have universal morphisms from every $X \in \mathrm{obj}(\boldsymbol{D})$ to R; let us name them $(X^*, i_X : X \to R(X^*))$.

Define hom-set bijection $\varphi_{X,A} : \mathrm{Hom}(X, R(A)) \to \mathrm{Hom}(X^*, A)$ defined as $f : X \to R(A)$ maps to $g : X^* \to A$ where g is the unique morphism from (X^*, i_X) to (A, f) in the comma category $(X \downarrow R)$. Observe that we have $R(g) \circ i_X = f$.

Define functor $L : \boldsymbol{D} \to \boldsymbol{C}$ with

- object map : $X \in \mathrm{obj}(\boldsymbol{D})$ maps to X^*, where $(X^*, i_X : X \to R(X^*))$ is a universal morphism from X to R.
- morphism map : $\left(A \xrightarrow{f} X\right) \mapsto \varphi\Big(\varphi^{-1}\Big(\mathrm{id}_{L(X)}\Big) \circ f\Big) = \varphi\Big(R\Big(\mathrm{id}_{L(X)}\Big) \circ i_X \circ f\Big) = \varphi(i_X \circ f)$.

This functor L is a left adjoint of R.

So there we have it; the hom-set bijection definition of R having a left adjoint is equivalent to having universal morphisms from every $X \in \mathrm{obj}(\boldsymbol{D})$ to R.

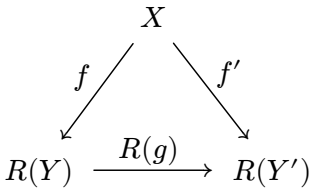
Now we shall approach the second question using our "newly discovered" universal morphism based definition of left adjoint.

We want to pack all this data together as a single universal morphism data. So ultimately, we want to get a comma category where the initial object will carry the data of all of these universal morphisms together.

So, in the comma category that we want, we want objects to carry the data

$$\left\{X \xrightarrow{f} R(Y) : X \in \mathrm{obj}(\boldsymbol{D}), Y \in \mathrm{obj}(\boldsymbol{C}), Y \text{ somehow depends on } X\right\},$$

and each morphism would be a *set* of commutative diagrams, each of which looks like



Observe that the objects of the comma category would be morphisms in the mediating category (the "playground" of the comma category construction).

Now, the data of the objects in the comma category (morphisms in the mediating category) seems awfully like the set of components of a natural transformation. So, morphisms in the mediating category are natural transformations from the functor $\mathrm{id}_{\boldsymbol{D}}$ to some other $\boldsymbol{D} \to \boldsymbol{D}$ functor.

Observe again the data of the objects in the comma category, Ys are dependent on the Xs somehow, through a functor perhaps? So, we want natural transformation from $\mathrm{id}_{\boldsymbol{D}}$ to functors of the form $R \circ -$.

So now, we consider the comma category $\mathrm{id}_{\boldsymbol{D}} \downarrow (R \circ -)$, where $R \circ -$ is a functor from $[D, C] \to [D, D]$, and $\mathrm{id}_{\boldsymbol{D}} \in [D, D]$ as expected. $(L, \tau : \mathrm{Nat}(\mathrm{id}_X, RL))$ is an initial object here where τ carries the data of the universal morphisms that we have as its components.

This natural transformation τ is known as the "unit".

Try again later. : Develop to include counit, then deploy the unit-counit definition.